6. Bölüm

Kontrol İşlemleri

6.1 Ardışık İşlem ve Kontrol İşlemleri

Yapısal Programlama başlığı altında programın ana fonksiyondan başlayacağı ve programlamanın ise birbirini çağıran fonksiyonlarla yapıldığı anlatılmıştı. Yapısal programlamada, ana fonksiyon dahil tüm fonksiyonlarda ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır. C++ dili de C diline eklenti yapılarak geliştirildiği için yapısal programlama mantığını iyi öğrenmeliyiz. Kal dı ki nesne yönelimli programdaki (object oriented programming) **nesne yöntemleri de yapısal programdaki fonksiyonlar gibi kodlanırlar.**

Şu ana karar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte **programın icrasını değiştirmeyen** bu tür talimatlara **ardışık işlem** (sequential operation) adı verilir.

Ardışık işlemler aşağıdaki üç tür talimattan (statement) oluşur.

1. Değişkenlerin kimliklendirildiği değişken tanımlamaları (identifier definition):
   (a) `int yariCap=3;`
   (b) `const float PI=3.14;`
   (c) `float daireninAlani,daireninCevresi;`
2. **İfadelerden** (expression): **İfade**, sabit, değişken ve işleç (operator) içeren söz dizimleridir;
   (a) `daireninAlani=PI*yariCap*yariCap;`
   (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (input-output operation):
   (a) `std::cout << "Kapasite Oranını (0.00-1.00) Giriniz:";`
   (b) `std::cin >> kapasiteOrani;`

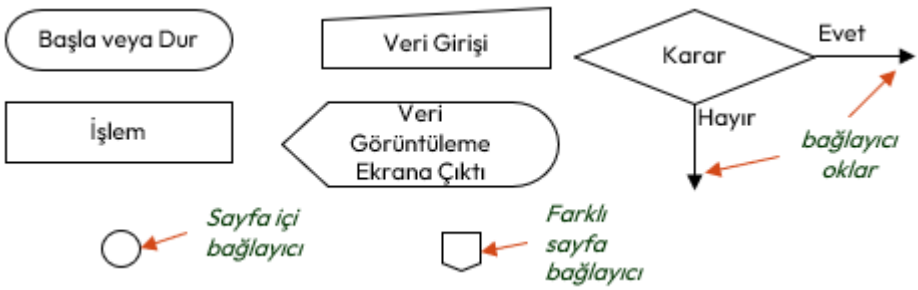
Kontrol işlemleri (control operation) ise programın icra sırasını değiştiren kontrol yapıları olup bu talimatlar (statement) olup üç türü vardır;

1. **Duruma göre seçimler** (conditional choice): `if`, `if-else` ve `switch` talimatları.
2. **İlişkisel döngü** (relational loop): `while`, `do-while` ve `for` talimatları.
3. **Dallanmalar** (jump): `continue`, `break`, `goto` ve `return` talimatları.

> Kontrol işlemlerinde; **kodlanan mantıksal satırlar (logical sequence) ile fiziksel olarak icra edilen satırlar (physical sequence) birbirinden farklıdır.**

6.2 Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.

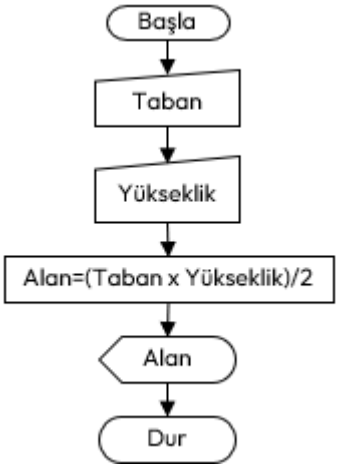


Şekil 6.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (flowchart), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde

kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



Şekil 6.2: Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- ◇ Akış şemalarında tek bir başlangıç simgesi olmalıdır
- ◇ Bitiş simgesi birden çok olabilir.
- ◇ Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- ◇ Bağlaç simgesi sayfanın dolmasından ötürü parçalanan akış şemasının öğelerini birleştirmede kullanılır.
- ◇ Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- ◇ Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

6.3 Duruma Göre Seçimler

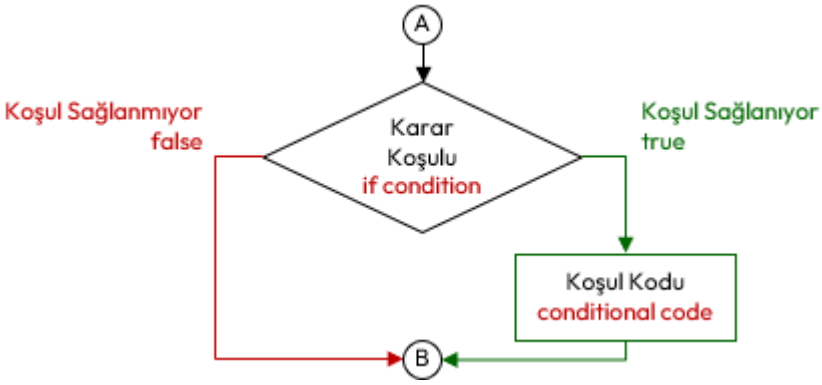
Duruma göre seçimler (conditional choice) programın akışını bir koşula göre değiştiren talimatlardır.

§6.3.1 If Talimatı

If talimatı, karar vermeye ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatının sözde kodu aşağıda verilmiştir;

```
if (karar-koşulu)
    koşul-kodu;
```

Burada ilişkisel işleçler içeren **karar koşulu** (condition) ifadesi (expression) test edilir. Sonuç `true` ise **koşul kodu** (conditional code) icra edilir, değilse icra edilmez. Bu talimatın akış diyagramı da aşağıdaki gibidir;



Şekil 6.3: If Talimatı İcra Akışı

Aşağıdaki örnek programı inceleyelim;

```
1 #include <iostream>
```

```
2  int main() {
3      int yas(18);
4      char cinsiyet='K';
5      if (yas<30)
6          std::cout << "Genç";
7      if (cinsiyet=='E')
8          std::cout << "Erkek";
9  }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. 1. Satırda `std::cout` akışı için `<iostream>` başlığı koda dahil edilmiştir.
2. 2. Satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. 3. ve 4. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. 5. Satırda bir `if` talimatı bulunmaktadır. **Karar koşulu**, `yas<30` ifadesiyle test edilecektir. Test sonucu `true` olduğundan 6. Satırdaki **koşul kodu** `std::cout << "Genç";` talimatı ile konsola `"Genç"` yazılır.
5. 7. Satırda da bir `if` talimatı bulunmaktadır. Karar koşulu, `cinsiyet=='E'` ifadesiyle test edilecektir. Test sonucu `false` olduğundan 8. Satırdaki koşul kodu talimatı **icra edilmez.**

If talimatında **koşul kodu**, her zaman yukarıdaki gibi **ardışık işlem** (sequential operation) olmaz. Bazen `if` gibi bir başka **kontrol işlemi** (control operation) de olabilir. Aşağıda **kademeli if** (compound if/cascade if) kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının koşul kodudur;

```
if (yas<30)
    if (yas<7) // Bu if, üsteki if talimatının koşul kodudur.
        std::cout << "Çoçuk";
```

Koşul kodu birden fazla talimattan oluşacak ise kod bloğu içine alınır. Aşağıda verilen kod örneğinde ilk `if` talimatında `yas<30` ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```
if (yas<30) { //koşul-kodu bloğu başlangıcı
    if (yas<7)
        std::cout << "Çocuk";
    if (yas<18)
        std::cout << "Genç";
    if (yas>60)
        std::cout << "Yaşlı";
} //koşul-kodu bloğu bitişi
```

Bazı durumlarda bir `if`, başka bir `if` bloğu içinde yani **iç içe** (nested if) olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```
if (cinsiyet=='E') { // dış if bloğu başlangıcı
    if (yas<7) { // iç if bloğu başlangıcı
        std::cout << "Erkek Çocuk ";
        std::cout << "Yada Erkek Bebek";
    } // iç if bloğu başlangıcı
    if (yas<18)
        std::cout << "Genç Delikanlı";
    if (yas>60)
        std::cout << "Yaşlı Adam";
} // dış if bloğu bitişi
```

If talimatında **koşul kodu**, her zaman tek bir test ifadesinden oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
if ((cinsiyet=='E') && (yas<18))  // hem Erkek, hem de genç
    std::cout << "Genç Delikanlı";
if ((cinsiyet=='K') && (yas>60)) // hem yaşlı, hem de kadın
    std::cout << "Yaşlı Kadın";
```

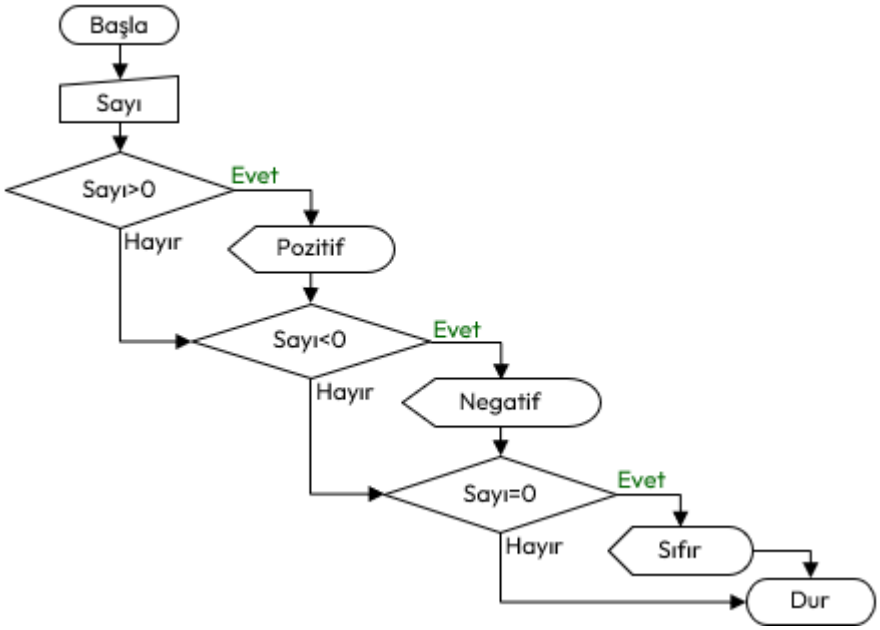
Ayrıca bazen programcı tarafından aşağıdaki gibi `if` talimatları yazabilir.

```
if (1) // veya if(true)
    std::cout << "Bu metin konsola her zaman yazılır.";
if (0) // veya if(false)
    std::cout << "Bu metin konsola hiçbir zaman yazılmaz!";
```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir C++ programı yazalım;

```
#include <iostream>
int main() {
    int sayi;
    std::cout << "Sayi Giriniz:";
    cin >> sayi;
    if (sayi>0) // sayının pozitif olup olmadığı test ediliyor.
        std::cout << "Pozitif"; // Burada sayının pozitif olduğu biliniyor.
    if (sayi<0) // sayının negatif olup olmadığı test ediliyor.
        std::cout << "Negatif"; // Burada sayının negatif olduğu biliniyor.
    if (sayi==0) // sayının sıfır olup olmadığı test ediliyor.
        std::cout << "Sıfır"; // Burada sayının sıfır olduğu biliniyor.
}
```

Bunun için oluşturacağımız akış diyagramı aşağıdaki şekilde olacaktır.



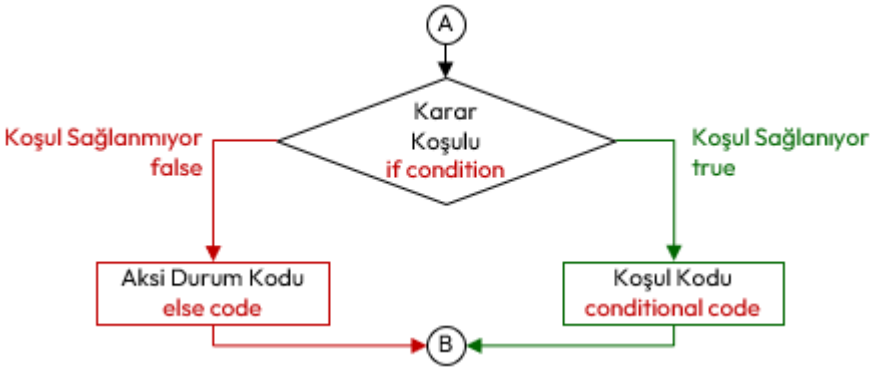
Şekil 6.4: Bir Sayının İşaretini Bulan Akış Diyagramı

§6.3.2 If-Else Talimatı

If-Else Talimatı, bir başka karar verme talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. Sözde kodu aşağıda verilmiştir;

```
if (karar-koşulu)
    koşul-kodu;
else
    aksi-durum-kodu;
```

If-Else talimatında, **karar koşulu** (condition) ifadesi `true` ise **koşul kodu** (conditional code) icra edilir, karar koşulu `false` ise **aksi durum kodu** (else code) icra edilir.



Şekil 6.5: If-Else Talimatı İcra Akışı

Aşağıda verilen örnek programı inceleyelim;

```
using namespace std;
int main() {
    int yas(65);
    char cinsiyet('K');
    if (yas<50)
        std::cout << "Genç ";
    else
        std::cout << "Yaşlı ";
}
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. 1. Satırda `std:cout` akışı için `<iostream>` başlığı koda dahil edilmiştir.
2. 2. satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. 3. ve 4. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. 5. Satırda bir `if` talimatı bulunmaktadır. **Koşul kodu** `yas<50` ifadesiyle test edilecektir. Test sonucu `false` olduğundan 6. Satırdaki **koşul kodu** talimatı **icra edilmez.** Bunun yerine `else` sonrası 8. Satırdaki **aksi durum kodu** talimatı **icra edilir.** Yani `std::cout << "Yaşlı ";` talimatı ile konsola `"Yaşlı"` yazılır.

If talimatında olduğu gibi bu talimatta da **koşul kodu** ya da **aksi durum kodu** birden fazla talimattan oluşacak ise kod bloğu içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

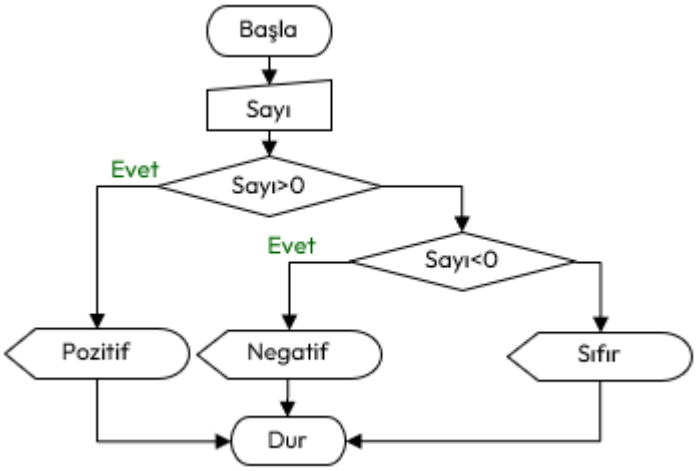
```
if (cinsiyet=='E') {
    if (yas<7)
        std::cout << "Erkek Çocuk ";
    if (yas<18)
        std::cout << "Genç Delikanlı ";
    if (yas>60)
        std::cout << "Yaşlı Adam ";
} else {
    if (yas<3)
        std::cout << "Kız Bebek ";
    if (yas<7)
        std::cout << "Kız Çocuk ";
    if (yas<18)
        std::cout << "Genç Kız ";
    if (yas>60)
        std::cout <<"Yaşlı Kadın ";
}
```

Bir sayının işaretini bulan C++ programının icra akışı olan Şekil 6.4 incelendiğinde ilk `if` talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılacaktır. Benzer şekilde ilk iki test geçilirse üçüncü if talimatındaki teste gerek yoktur. Bu durumu Şekil 6.6'de verilen akış diyagramında daha net görebiliriz. Bu durumda aynı program if-else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```
1  #include <iostream>
2  int main() {
3      int sayi;
4      std::cout << "Sayi Giriniz:";
5      cin >> sayi;
6      if (sayi>0) { // sayının pozitif olup olmadığı test ediliyor.
7          std::cout << "Pozitif"; // Burada sayının pozitif olduğu biliniyor.
8      } else {  // Burada sayının pozitif olmadığı biliniyor.
9          if (sayi<0) { // sayının negatif olup olmadığı test ediliyor.
10             std::cout << "Negatif"; // Burada sayının negatif olduğu biliniyor.
11         } else  { // Burada sayının pozitif ve negatif olmadığı test biliniyor.
12             std::cout << "Sıfır"; // Burada sayının sıfır olduğu biliniyor.
13         }
14     }
15 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- Eğer `sayı` pozitif girilirse;
  - 6. Satırdaki `if` karar koşulunda `sayi>0` test edilip `true` çıkacak ve sadece 7. Satırdaki koşul kodu olan `std::cout << "Pozitif";` talimatı icra edilecektir. 8. Satırdan başlayan ve on üçüncü satırda



Şekil 6.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

biten aksi durum kodu icra edilmeyecektir.

- Eğer `sayı` negatif girilirse;
  - 6. Satırdaki `if` karar koşulunda `sayi>0` test edilip `false` çıkacak ve 8. Satırdan başlayan ve 14. Satırda biten aksi durum kodu icra edilecektir.
  - Bu blok içinde 9. Satırdaki `if` karar koşulu `sayi<0` test edilip `true` çıkacak ve 10. Satırdaki koşul kodu olan `std::cout << "Negatif";` talimatı icra edilecektir. 12. Satırlardaki aksi durum kodu icra edilmeyecektir.
- Eğer `sayı` sıfır girilirse;
  - 6. Satırdaki `if` karar koşulunda `sayi>0` test edilip `false` çıkacak ve 8. Satırdan başlayan ve 14. Satırda biten aksi durum kodu icra edilecektir.
  - Bu blok içinde 9. Satırdaki `if` karar koşulu `sayi<0` test edilip `false` çıkacak ve 10. Satırdaki koşul kodu talimatı icra edilemeyecektir. Onun yerine on 12. Satırlardaki aksi durum kodu olan `std::cout << "Sıfır";` icra edilecektir.

Ayrıca kod incelendiğinde her bir `if` ve `else` sonrasında tek talimat bulunmaktadır. Dolayısıyla blok kullanmaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```
#include <iostream>
int main() {
    int sayi;
    std::cout << "Sayi Giriniz:";
    cin >> sayi;
    if (sayi>0)
        std::cout << "Pozitif";
    else
        if (sayi<0)
            std::cout << "Negatif";
        else
            std::cout << "Sıfır";
}
```

§6.3.3 Sarkan Else

Aşağıdaki program örneği incelendiğinde `else`, hangi `if` talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz. Bu problem **sarkan else** (dangling else) problemi olarak bilinir.

```
if (cinsiyet=='E') if (yas<18) std::cout << "Delikanlı"; else std::cout << "Adam";
```

Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk `if` talimatının **koşul kodunun** ikinci if-else talimatı olduğu görülmektedir.

```
if (cinsiyet=='E')
    if (yas<18)
        std::cout << "Delikanlı";
    else
        std::cout << "Adam";
```

Kod bloğu kullanılmadan yazılan bu tip kodlarda `else` her zaman kendinden bir önceki `if` talimatına aittir.

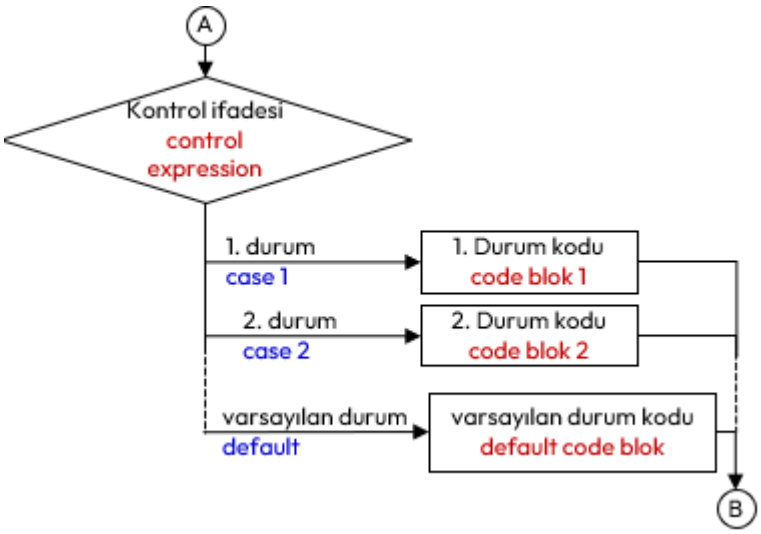
Sarkan else durumundaki mantıksal hataların önüne geçmek için
acemi kullanıcılar her zaman blok kullanmalıdır.

```
 if (cinsiyet=='E') {
     if (yas<18) {
         std::cout << "Delikanlı";
     } else {
         std::cout << "Adam";
     }
}
```

§6.3.4 Switch Talimatı

Switch talimatı, bir **kontrol ifadesi** (control expression) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```
switch (kontrolifadesi) { //switch bloğu başlangıcı
    case alternatif-1:
        alternatif-1-kodu;
        break; //break talimatı zorunlu değildir
    case alternatif-2:
        alternatif-2-kodu;
        break; //break talimatı zorunlu değildir.
    // ...
    default: // Bu etiket zorunlu değildir!
        varsayılan-alternatif-kodu;
} //switch bloğu bitişi
```



Şekil 6.7: Switch Talimatı İcra Akışı

Switch talimatına ilişkin kurallar;

1. **Kontrol ifadesi, sonucu tam sayı olan ifadedir** ve **bir kez değerlendirilir**.
2. **Kod bloğu olmayan** bir `switch` talimatı **yazılamaz!**
3. **Her bir alternatif**, bir **tam sayı değişmezi** (integer literal) izleyen ve iki nokta ile biten bir **etiket (label) olarak tanımlanır.**
4. **Alternatif**, hiçbir zaman **değişken veya ifade (expression) olamaz!**
5. `switch` **bloğu icra edilirken**, karşılaştığı ilk `break;` talimatında, `switch` bloğu **dışına çıkılır**. Bu talimat zorunlu değildir.
6. Tüm tam sayıları içerecek şekilde alternatiflerin tümü yazılmayabilir. Blok sonunda yer alacak şekilde **üzerinde yer alan alternatifler dışında kalan tüm alternatifler için** `default:` etiketli alternatif yazılabilir. Bu etiket zorunlu değildir.

Aşağıda örnek olarak menü seçimi için yazılmış bir program örneği verilmiştir;

```
1 #include <iostream>
```

```
2  int main() {
3      int menu;
4      std::cout << "Menü İçin Rakam Giriniz:";
5      std::cin >> menu;
6      switch(menu) {
7          case 1:
8              std::cout << "1 Numaralı Menüyü Seçtiniz.";
9              std::cout << "Hamburger ve Ayran Hazırlanacak.";
10             break;
11         case 2:
12             std::cout << "2 Numaralı Menüyü Seçtiniz.";
13             std::cout << "Patates Kızartması ve Kola Hazırlanacak.";
14             break;
15         case 3:
16         case 4:
17             std::cout << "3 veya 4 Numaralı Menüyü Seçtiniz.";
18             break;
19         default:
20             std::cout << "1,2, 3 veya 4 Dışında Menü Seçtiniz.";
21             std::cout << "Böyle bir Menümüz Yok!.";
22     }
23 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- Eğer `menu`, `1` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 1 olduğuna karar verilir.
  - Bu durumda 7. Satırdaki `case 1:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 8. Satırdaki `std::cout << "1 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 9. Satırdaki `std::cout << "Hamburger ve Ayran Hazırlanacak.";` icra edilir ve bir sonraki talimata geçilir.
    - Son olarak 10. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`, `2` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 2 olduğuna karar verilir.
  - Bu durumda 11. Satırdaki `case 2:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 12. Satırdaki `std::cout << "2 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 13. Satırdaki `std::cout << "Patates Kızartması ve Kola Hazırlanacak.";` icra edilir ve bir sonraki talimata geçilir.
    - 14. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`, `3` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 3 olduğuna karar verilir.
  - Bu durumda 15. Satırdaki `case 3:` etiketine gidilir ve bu alternatife ilişkin yazılmamıştır. Ancak ilk icra edilebilir koda kadar ilerlenir ve oradaki kodlar icra edilir;
    - İlk önce 17. Satırdaki `std::cout << "3 veya 4 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 18. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`, `4` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 4 olduğuna karar verilir.
  - Bu durumda 16. Satırdaki `case 4:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 17. Satırdaki `std::cout << "3 veya 4 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 18. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`,`-1` , `0`, `12` ve `300` gibi yukarıdaki alternatiflerden farklı girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu düzenlenememiş bir durum olduğuna karar verilir.
  - Bu durumda 19. Satırdaki `default:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;

- İlk önce 20. Satırdaki `std::cout << "1,2, 3 veya 4 Dışında Menü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
- Daha sonra 21. Satırdaki `std::cout << "Böyle bir Menümüz Yok!.";` talimatı bizi `switch` icra edilir. Artık `switch` bloğunun sonuna ulaşıldığından bloktan çıkılır.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalan üzerinden işlem yapan `switch` talimatı bir başka örnek program verilmiştir;

```
#include <iostream>
int main() {
    int sayi;
    std::cout << "Bir Sayı Giriniz:";
    std::cin >> sayi;
    switch(sayi%4) { //konrol ifadesi bir işlem içerir
        case 3:
            std::cout << "Sayı 4 rakamına bölündüğünde kalan 3 dir.";
            break;
        case 2:
            std::cout << "Sayı 4 rakamına bölündüğünde kalan 2 dir.";
            break;
        case 1:
            std::cout << "Sayı 4 rakamına bölündüğünde kalan 1 dir.";
            break;
        default: // Başka alternatif yoktur.
            std::cout << "Sayı 4 Rakamına TAM Bölünür";
    }
}
```

§6.3.5 Üçlü Koşul İşleci

Daha önce İşleçler başlığı altında işlenenlere ek olarak, **ardışık işlemlerde** (sequential operation) if talimatlarına gerek kalmadan bir koşula göre **ifadeler** (expression) işlenecek ise **üçlü koşul işleci** (conditional tenary operator) kullanılır. Aşağıda üçlü işlecin iki sözdizimi verilmiştir;

```
koşul ? doğruifadesi : yanlışifadesi;
değişken = koşul ? doğruifadesi : yanlışifadesi;
```

Aşağıda oy kullanma durumu örneği bu işleçle işlenmiştir;

```
#include <iostream>
int main() {
    int yas;
    std::cout << "Yaşınız?: ";
    std::cin >> yas;
    (yas >= 18) ? std::cout << "Oy Kullanabilirsin." : std::cout << "Oy Kullanamazsın!";
}
```

Aşağıda başka kullanımlarına ilişkin farklı kod örneği verilmiştir;

```
#include <iostream>
int main() {
    int sayi,tek,cift;
    std::cout << "Bir Sayı Giriniz: ";
    std::cin >> sayi;
    tek= (sayi%2) ? 1 : 0;
    cift= tek ? 0 : 1;
    int sonuc1=tek ? sayi+30 : sayi+40;
    int sonuc2=cift ? sayi*30 : sayi*40;
}
```

6.4 İlişkisel Döngüler

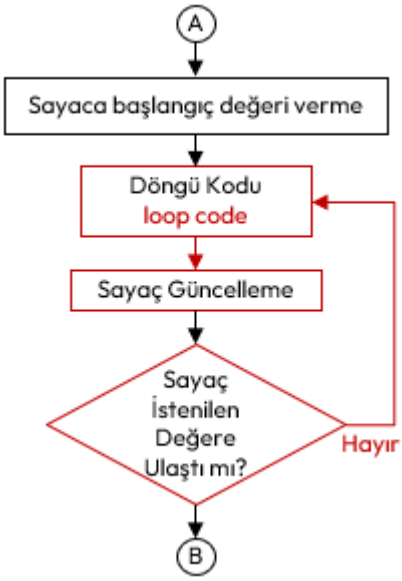
İşlemciler iyi bir sayıcıdırlar. CPU içindeki kaydediciler (registers) sayma işlevini ve birçok matematiksel işlemi yapmak için de kullanılır. **Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir.** Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım.

```
#include <iostream>
int main() {
    std::cout << "ILHAN" << endl;  //1
    std::cout << "ILHAN" << endl;  //2
    std::cout << "ILHAN" << endl;  //3
    std::cout << "ILHAN" << endl;  //4
    std::cout << "ILHAN" << endl;  //5
    std::cout << "ILHAN" << endl;  //6
    std::cout << "ILHAN" << endl;  //7
    std::cout << "ILHAN" << endl;  //8
    std::cout << "ILHAN" << endl;  //9
    std::cout << "ILHAN" << endl;  //10
}
```

Program incelendiğinde aynı `std::cout` talimatının tekrar tekrar yazıldığını görürüz. **Kod tekrarı istenmeyen bir durumdur!**

§6.4.1 Sayaç Kontrollü Döngüler

Belirlenen bir sayaç değişkeninin istenilen bir değere ulaşıp ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır. Yapısal programlama öncesinde tekrar edilecek kodun başına dönüş `goto` talimatıyla sağlanır. **Bu talimat icra sırasını etkileyen bir talimattır.** C++ dilinde Kodlama sırasında tekrar edilecek



Şekil 6.8: Sayaç Kontrollü Döngü İcra Akışı

kodun başına iki nokta ile biten bir **etiket** (label) konulur. Etiketlere kimlik verilirken yine 4.3.5 başlığındaki değişken kimliklendirme (identifier definition) kuralları uygulanır. Tekrar edilecek kodun sonunda ise `goto` talimatı etiketle birlikte kullanılır.

```
1 #include <iostream>
2 int main() {
3     int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
4     Etiket: //Tekrar Edilecek Kodun Başı ETİKETLENİR. Etilet kimliği "Etiket"
5         std::cout << "ILHAN" << endl;
6         sayac++; //Sayaç Güncelleme
7     if (sayac<10) //Sayaç Kontrolü
8         goto Etiket; // İstenilen değere ulaşılmamış ise etikete git.
9 }
```

Örneği şu şekilde açıklayabiliriz;

- 3.Satırda tekrar edilecek koda girmeden sayacımıza ile değer verdik.
- 4.Satıra geri dönülecek kodun başını işaretleyen bir etiket konulmuştur.
- 5.Satırda tekrarlanacak talimat yazılmıştır. Birden fazla talimat da yazabiliriz.
- 6.Satırda sayacımızı güncelledik.

- ◇ 7.Satırda sayacın istenilen değere ulaşıp ulaşmadığını kontrol ettik. Eğer ulaşmamış ise 8.Satırdaki `goto Etiket;` talimatıyla 4.Satıra döndük.

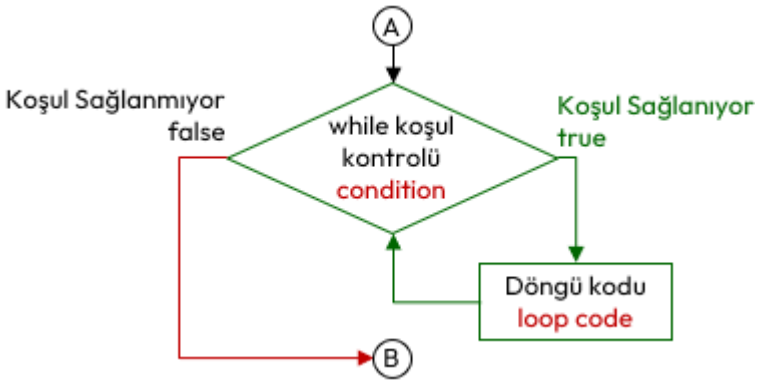
Sonuç olarak 5. ile 8. Satır arasında dört talimat içeren **mantıksal satır** (logical sequence) olmasına karşın 40 adet **fiziksel satır** (physical sequence) icra edilmiştir. C++ programlama dili, donanıma yakın bir dil olduğundan `goto` talimatı desteklenir. Ancak 1968 Yılında *Edsger W. Dijkstra* tarafından `goto` ifadelerinin zararlı olduğunu ilan ettiği unutulmamalıdır.

> Yukarıda verilen döngü kodu örneğindeki gibi arapsaçı kodlamadaki `goto` talimatı yerine, **yapısal programlamada** (structural programming) ilişkisel döngü (relational loop) talimatları olan `while`, `do-while` ve `for` talimatları geliştirilmiştir.
> **C++ dilinde `goto` talimatının kullanımı önerilmez!**

§6.4.2 While Talimatı

While döngüsünde **koşul** (condition), `true` olduğu sürece **döngü kodu** (loop code) icra edilir. Tekrarlanacak kod, tek bir talimat olabileceği gibi bir kod bloğu da olabilir. While döngüsünün sözde kodu aşağıda verilmiştir;

```
while (koşul)
    döngü-kodu;
```



Şekil 6.9: While Talimatı İcra Akışı

While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
#include <iostream>
int main() {
    int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
    while (sayac<10) // While koşul kontrolü
    {  // döngü bloğu başlangıcı
        std::cout << "ILHAN" << endl;
        sayac=sayac+1; //Sayaç Güncelleme
    }  // döngü bloğu bitişi
}
```

Sayaç güncelleme ifadesi, while **koşul** ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır;

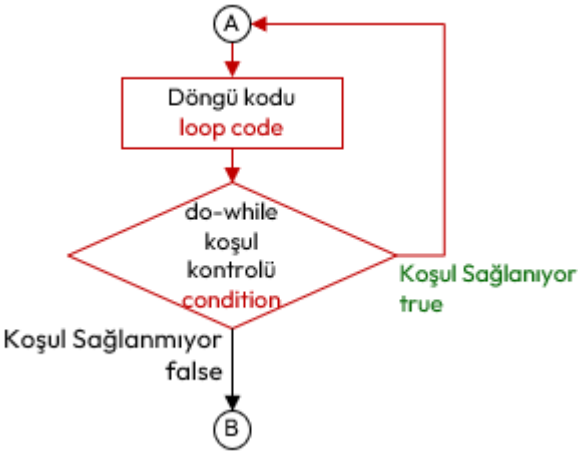
```
#include <iostream>
int main() {
    int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
    while (sayac++<10) // hem sayaç güncelleme hem de koşul kontrolü
        std::cout << "ILHAN" << endl; // tek talimat olduğundan blok kullanılmadı
}
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

§6.4.3 Do-While Talimatı

Do-While döngüsünde, `do` ile `while` saklı kelimeleri arasındaki **döngü kodu** (loop code), **koşul** (condition) `true` olduğu sürece tekrarlanarak icra edilir. Tekrarlanacak kod tek bir talimat olabileceği gibi bir kod bloğu da olabilir. Do-While talimatının sözde kodu aşağıda verilmiştir;

```
do   // "do" kelimesi goto talimatındaki etiket gibi davranır.
 döngü-kodu;
while (koşul);
```



Şekil 6.10: Do-While Talimatı İcra Akışı

Do-While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
#include <iostream>
int main() {
    int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
    do {  // döngü bloğu başlangıcı
        std::cout << "ILHAN" << endl;
        sayac=sayac+1; //Sayaç Güncelleme
    } // Döngü bloğu bitişi
    while (sayac<10); // Do-While koşul kontrolü
}
```

Sayaç güncelleme ifadesi, **koşul** ifadesine eklenerek program aşağıdaki şekilde daha da kısaltılabilir.

```
#include <iostream>
int main() {
    int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
    do
        std::cout << "ILHAN" << endl;
    while (sayac++<10); // Hem koşul kontrolü hem de sayaç güncelleme
}
```

Yukarıdaki son örnek incelendiğinde ibir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

While döngüsünde koşul kontrolü en başta yapılır!
Do-While döngüsünde ise döngü bloğu en az bir kez çalıştırıldıktan sonra koşul kontrolü yapılır!

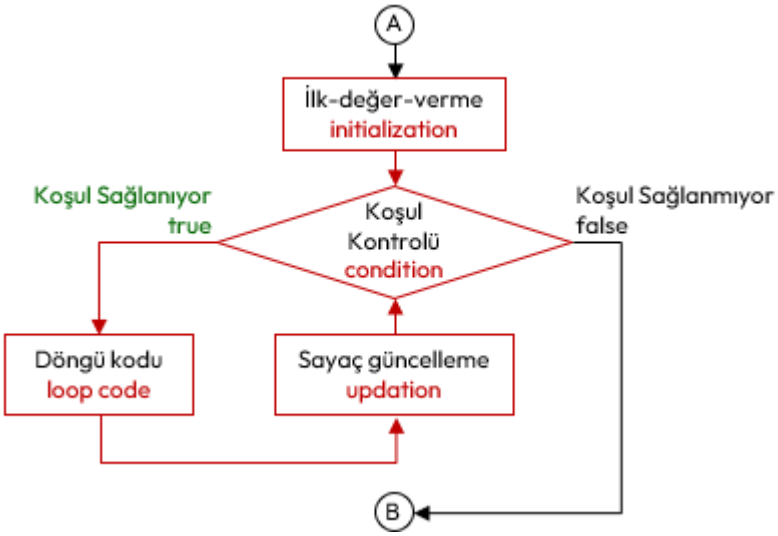
§6.4.4 For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```

For talimatı aşağıdaki şekilde icra edilir;

1. **İlk değer verme ifadesi bir kez icra edilir!**
2. **Koşul kontrolü yapılır!** Eğer test sonucu `true` ise döngü kodu icrasına geçilir. Aksi halde döngüden çıkılır.
3. **Döngü kodu** icra edilir.
4. Döngü kodu icrası bitince **sayaç güncellemeleri yapılır**.
5. Tekrar ikinci adımdaki **koşul kontrolüne dönülür**.



Şekil 6.11: For Talimatı İcra Akışı

For talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir.

```
#include <iostream>
int main() {
    for (int sayac=1; sayac<10; sayac++) { //1. mantıksal satır.
        std::cout << "ILHAN" << std::endl; //2. mantıksal satır
    }
}
```

Buradaki döngü mantıksal satır (logical sequence) olarak iki talimattan oluşmaktadır. Ancak fiziksel satır (physical sequence) olarak 10 satır icra edilmiştir.

Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program örneği gösterilmektedir.

```
1  #include <iostream>
2  #include <iomanip> // setw(), setfill() için
3  int main() {
4      for (int sayac=1; sayac<11; sayac++)
5          std::cout << std::setw(2) << std::setfill('0') << sayac << ". ILHAN" << std::endl;
6  }
7  /*Program Çıktısı:
8  01. ILHAN
9  02. ILHAN
10 03. ILHAN
11 04. ILHAN
12 05. ILHAN
13 06. ILHAN
14 07. ILHAN
15 08. ILHAN
16 09. ILHAN
17 10. ILHAN
18 */
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu 4.Satırda belirtilen bir adet talimat içermektedir. Mantıksal olarak 1 satır içermesine rağmen 10 adet fiziksel satır icra edilmiştir.

For döngüsü, bir sayaç ile çalışılabileceği gibi **birden fazla sayaç ile de çalışılabilir**. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işleci** (comma operator) kullanılır.

```
#include <iostream>
#include <iomanip> // setw(), setfill() için
int main() {
    for (int sayac1=1, sayac2=30; sayac1<11; sayac1++,sayac2--)
```

```
    std::cout << std::setw(2) << std::setfill('0') << sayac1 << "-" << sayac2 << ". ILHAN" <<
     ↪  std::endl;
}
/*Program Çıktısı:
01-30. ILHAN
02-29. ILHAN
03-28. ILHAN
04-27. ILHAN
05-26. ILHAN
06-25. ILHAN
07-24. ILHAN
08-23. ILHAN
09-22. ILHAN
10-21. ILHAN
*/
```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```
#include <iostream>
int main() {
    int sayac, sayi1,sayi2;
    int tekToplam=0,ciftToplam=0;
    std::cout << "İki Sayı Giriniz: ";
    std::cin >> sayi1 >> sayi2;
    if (sayi2<sayi1) { //sayi2, sayi1 den büyük olmalı.
        int temp=sayi1; //küçük ise yer değiştiriyoruz.
        sayi1=sayi2;
        sayi2=temp;
    }
    for (sayac=sayi1; sayac<=sayi2; sayac=sayac+1) {
        if (sayac%2==1) //sayac tek mi?
        tekToplam+=sayac;
        else
        ciftToplam+=sayac;
    }
    std::cout << "Tek Toplam:" << tekToplam << " Çift Toplam:" << ciftToplam;
}
/* Program Çıktısı:
İki Sayı Giriniz: 9 17
Tek Toplam: 65 Çift Toplam: 52
*/
```

Bir ve kendisinden başka tam böleni olmayan sayıya **asal sayı** denir. Klavyeden girilen pozitif bir tam sayının asal olup olmadığını ekrana yazdıran C++ programı aşağıdaki şekilde kodlanabilir;

```
#include <iostream>
int main() {
    int sayi,asal=1;
    std::cout << "Bir Sayı Giriniz:";
    std::cin >> sayi;
    for (int sayac=sayi-1; (sayac>1)&&(asal==1); sayac--)
        if (sayi%sayac==0)
            asal=0;
    if(asal)
        std::cout << "Girilen " << sayi << " sayısı asaldır.";
    else
        std::cout << "Girilen " << sayi << " sayısı asal değildir.";
}
```

Klavyeden girilen pozitif bir tam sayının **asal çarpanlarını** ekrana yazdıran C++ programı aşağıdaki şekilde kodlanabilir;

```
#include <iostream>
int main() {
    int sayi;
```

```
    std::cout << "Bir Sayı Giriniz:";
    std::cin >> sayi;
    for (int sayac=sayi; sayac>0; sayac--)
    if (sayi%sayac==0)
    std::cout << "Asal çarpan:" << sayac <<std::endl;
}
/* Program Çıktısı:
Bir Sayı Giriniz:12
Asal çarpan:12
Asal çarpan:6
Asal çarpan:4
Asal çarpan:3
Asal çarpan:2
Asal çarpan:1
*/
```

§6.4.5 İç İçe Döngü Kodu

Döngüler içinde yer alan döngü kodu bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde **iç içe döngü** (nested loop) kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk `for` döngüsünün tekrarlanan kodu ikinci bir `for` döngüsüdür.

```
#include <iostream>
int main() {
    int satir,sutun;
    for (satir=0; satir<5; satir++) {
        for (sutun=0; sutun<4; sutun++)
        std::cout << "x";
        std::cout << std::endl;
    }
}
/* Program Çıktısı:
xxxx
xxxx
xxxx
xxxx
xxxx
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program aşağıda verilmiştir;

```
#include <iostream>
#include <iomanip>
int main() {
    int satir,sutun;
    //Başlığı Yazdıran Kısım
    std::cout << "   Sütun: ";
    for(sutun=0; sutun<3;sutun++)
    std::cout << std::setw(3) << sutun+1 << " ";
    std::cout << std::endl;

    //Matrisi Yazdıran Kısım
    for (satir=0; satir<5; satir++) { //Satır için
        std::cout << std::setw(2) << std::setfill('0')  << satir+1 << ".Satir: "; //İmleç,
        ↪ yazdırılan metnin sonunda olacaktır.
        for(sutun=0; sutun<3;sutun++) //Sütun için
        std::cout << std::setw(3) << std::setfill('0') <<sutun << " "; //İmleç, yazdırılan metnin
        ↪ sonunda olacaktır.
        std::cout << std::endl; //Satır işi bitince imleci yeni satıra geçir!
    }
}
/* Program Çıktısı:
   Sütun:   1   2   3
```

```
01.Satir: 000 001 002
02.Satir: 000 001 002
03.Satir: 000 001 002
04.Satir: 000 001 002
05.Satir: 000 001 002
*/
```

§6.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (sentinel value) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden `0` girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
#include <iostream>
int main() {
    int sayi; /* okunan tam sayı */
    int toplam(0); /* girilen sayıların toplamı. başlangıçta 0*/
    do {
        std::cout << "Bir sayı giriniz (Bitirmek için 0): ";
        std::cin >> sayi;
        toplam+=sayi; // sayı 0 girilse bile toplam etkilenmez
    } while (sayi != 0); //sentinel value 0

    std::cout << "Girilen Sayıların Toplamı:" << toplam;
}
```

Aşağıda pozitif sayı girilmesini sağlayan bir başka kod örneği verilmiştir.

```
#include <iostream>
int main() {
    int sayi,pozitifSayi(0);
    do { /* Pozitif girilmesini zorluyoruz */
        std::cout << "Lütfen pozitif sayı giriniz: ";
        std::cin >> sayi;
        if (sayi<=0)
        st6d::cout << "HATA:Pozitif Sayı GİRMEDİNİZ!" << std::endl;
    } while (sayi <= 0);
    pozitifSayi=sayi;
    std::cout <<"Girilen pozitif tam sayı: " << pozitifSayi;
}
```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir. Ancak burada 4. ve 5. satırlar ile 8. ve 9. satırların aynı olduğuna ve kod tekrarı yapıldığına dikkat ediniz!

```
1  #include <iostream>
2  int main() {
3      int sayi,pozitifSayi(0);
4      std::cout << "Lütfen pozitif sayı giriniz: ";
5      std::cin >> sayi;
6      while (sayi <= 0) {
7          std::cout << "HATA:Negatif sayı giriniz!" << endl;
8          std::cout << "Lütfen pozitif sayı giriniz: ";
9          std::cin >> sayi;
10     }
11     pozitifSayi=sayi;
12     std::cout <<"Girilen pozitif tam sayı: " << pozitifSayi;
13 }
```

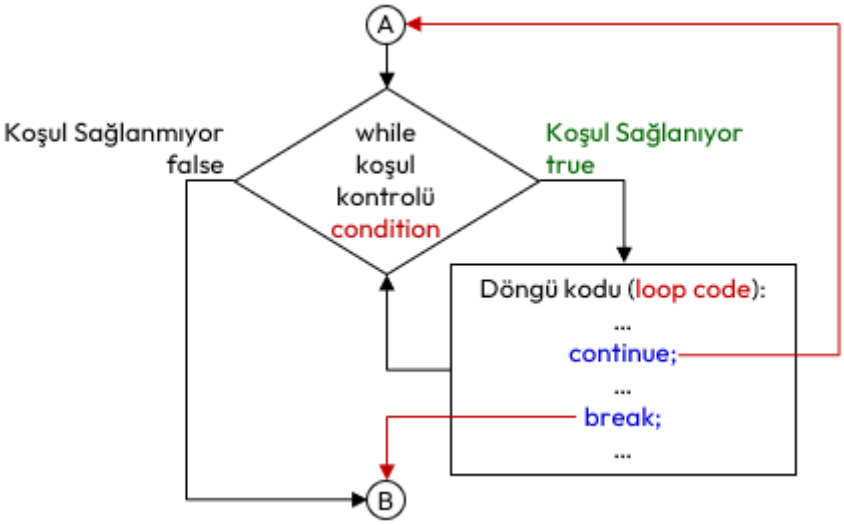
6.5 Dallanmalar

Dallanmalar (jump), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

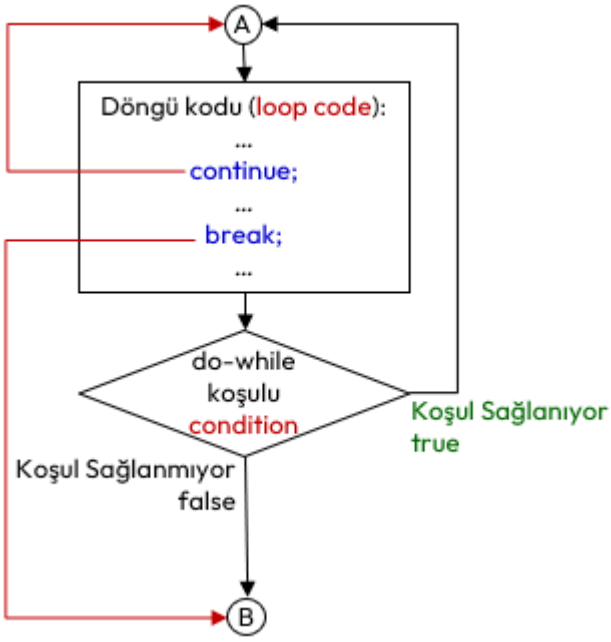
§6.5.1 Continue ve Break Talimatları

Continue talimatı , yalnızca geçerli **yinelemeyi** (iteration) sonlandırır ve sonraki yinelemelerle devam eder. Yani bu talimat, bir sonraki yinelemeyi yapmak için kullanılır. Dolayısıyla sadece `while`, `do-while` ve `for` **döngü kodları** (loop code) içinde kullanılabilir.

Break talimatı , döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bunu daha önce `switch` talimatında görmüştük. Bu talimat, aynı şekilde `while`, `do-while` ve `for` döngüleri ile kullanılabilir. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 6.12'de verilmiştir.



(a) While



(b) Do-While

Şekil 6.12: While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı

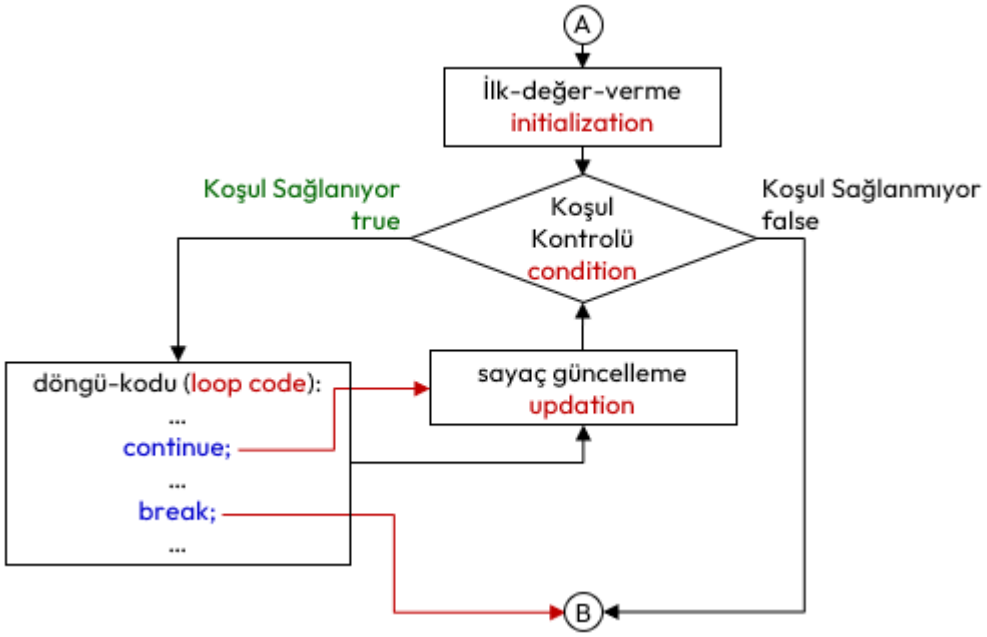
Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz. Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

```
#include <iostream>
int main() {
    int i;
    for (i=0; i<=15; i=i+1) {
        if (i<7) //i'nin değeri 7 den küçük ise
            continue;  //bir sonraki yinelemeye (iteration) geç

        if (i==10) //i'nin değeri 10 ise
            continue;  //bir sonraki yinelemeye (iteration) geç
```

```
        if (i%5==0) // i'nin değeri 5 in katı ise
            continue;  //bir sonraki yinelemeye (iteration) geç
        std::cout << "i=" << i << std:endl;
    }
}
/*Program Çıktısı:
i = 7
i = 8
i = 9
i = 11
i = 12
i = 13
i = 14
*/
```

Yukarıdaki örnek incelendiğinde döngü bloğu üç adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 42 adet fiziksel satır (physical sequence) icra edilmiştir. Bazı `continue;` talimatları `std::cout` talimatının icrasını engellemiştir.



Şekil 6.13: For Döngüsünde Break ve Continue Talimatları İcra Akışı

Aşağıda pozitif sayı girilmesini zorlayan **sonsuz döngü** (infinitive loop) içeren program verilmiştir. Programda döngü, pozitif sayı girildiğinde `break;` talimatı ile kırılmaktadır.

```
#include <iostream>
int main() {
    int sayi, pozitifSayi(0);
    while (1) /* Sonsuz döngü */
    {
        std::cout << "Lütfen pozitif sayı giriniz: ";
        std::cin >> sayi;
        if (sayi<=0)
            std::cout << "HATA:Pozitif Sayı GİRMEDİNİZ!" << std::endl;
        else
            break; //döngüden çık
    }
    pozitifSayi=sayi;
    std::cout << "Girilen pozitif tam sayı: " << pozitifSayi;
}
```

Bu talimatları dikkatli kullanmalıyız. Bazı durumlarda sonsuz döngüye girme durumu tüm döngü talimatlarında yaşanabilir. `for` Döngüsünde bu durum farklıdır. `continue;` talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır.

§6.5.2 Return Talimatı

Yapısal programlamada **ana fonksiyonun** (main function) sonunda çokça kullandığımız bu talimat, bir fonksiyonun üreteceği ya da belirlenen değeri geri döndürür. Kullanıldığı yerden fonksiyon bloğu dışına çıkılır. Aşağıda üç defa negatif sayı girilmesi halinde programı sonlandıran bir program verilmiştir.

```
#include <iostream>
int main() {
    int sayi;
    int pozitifSayi=0;
    int sayac=0; //kaç defa pozitif olmayan sayı girildi.
    do { // Pozitif girilmesini zorluyoruz
        std::cout << "Lütfen pozitif sayı giriniz: ";
        std::cin >> sayi;
        if (sayi<=0) {
            std::cout <<"HATA: Pozitif Sayı GİRMEDİNİZ!" << std::endl;
            sayac++;
            if (sayac==3) {
                std::cout << "Üç defa negatif sayı girdiniz. ";
                std::cout << "Programdan Çıkılıyor..." << std::endl;
                return 1; // ana fonksiyondan çıkılarak işletim sistemine 1 gönderiliyor.
            }
        }
    } while (sayi <= 0);
    pozitifSayi=sayi;
    std::cout << "Girilen pozitif tam sayı: " << pozitifSayi;
    return 0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -2
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -3
HATA: Pozitif Sayı GİRMEDİNİZ!
Üç defa negatif sayı girdiniz. Programdan Çıkılıyor.
*/
```

§6.5.3 Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren `goto` talimatıdır. 6.4.1 başlığında örneği verilmiştir.

6.6 Düşün ve Çöz

- **Düşün**: Sarkan else (dangling else) problemi nedir? Bu problemi önlemek için neden her zaman süslü parantez kullanılması önerilir?
- **Düşün**: While ve do-while döngüleri arasındaki farkı, bir kullanıcıdan şifre isteme senaryosu üzerinden karşılaştırınız.
- **Düşün**: Switch talimatında `break;` kullanılmadığında oluşan başarısız olma (fall-through) durumunu bir hata değil, bir özellik olarak nasıl kullanabiliriz?
- **Çöz**: Kullanıcıdan sürekli sayı alan ve `0` girildiğinde o ana kadar girilen en büyük sayıyı ve sayıların ortalamasını ekrana basan bir gözcü kontrollü döngü (sentinel value) tasarımı yapınız.
- **Çöz**: İç içe döngüler kullanarak ekrana 1'den 10'a kadar olan sayıların çarpım tablosunu düzgün bir formatta yazdırınız.
- **Çöz**: Kullanıcı negatif bir sayı girene kadar sayı alan ve girilen tek sayıların toplamını, çift sayıların ise adedini ekrana basan programı yazınız.